

IMPERIAL COLLEGE LONDON

TIMED REMOTE ASSESSMENTS 2020-2021

BEng Honours Degree in Computing Part II
MEng Honours Degrees in Computing Part II
BEng Honours Degree in Mathematics and Computer Science Part II
MEng Honours Degree in Mathematics and Computer Science Part II
for Internal Students of the Imperial College of Science, Technology and Medicine

*This paper is also taken for the relevant assessments for the
Associateship of the City and Guilds of London Institute*

PAPER COMP50004

OPERATING SYSTEMS

Friday 7 May 2021, 10:00

Duration: 105 minutes

Includes 15 minutes for access and submission

Answer ALL TWO questions

Open book assessment

This time-limited remote assessment has been designed to be open book. You may use resources which have been identified by the examiner to complete the assessment and are included in the instructions for the examination. You must not use any additional resources when completing this assessment.

The use of the work of another student, past or present, constitutes plagiarism. Giving your work to another student to use constitutes an offence. Collusion is a form of plagiarism and will be treated in a similar manner. This is an individual assessment and thus should be completed solely by you. The College will investigate all instances where an examination or assessment offence is reported or suspected, using plagiarism software, vivas and other tools, and apply appropriate penalties to students. In all examinations we will analyse exam performance against previous performance and against data from previous years and use an evidence-based approach to maintain a fair and robust examination. As with all exams, the best strategy is to read the question carefully and answer as fully as possible, taking account of the time and number of marks available.

Paper contains 2 questions

- 1 A 64-bit virtual memory system uses a 4-level page table with 4 KB pages. Its *translation look-aside buffer* (TLB) takes 0.5 ns to access. A memory access takes 15 ns.
- a Describe the purpose of a TLB and list the information that is stored in its entries.
 - b Draw a diagram that shows how a TLB fits into the memory hierarchy on a **multicore** CPU.
 - c Calculate the *hit ratio* α that the TLB must achieve so that the *effective access time* (EAT) of the above virtual memory system is 20 ns. Show your calculation.
 - d Explain what happens to the TLB (and the associated performance implications) when the OS scheduler performs the following switches:
 - i) from one process to another process, running on the same CPU core
 - ii) from one thread to another thread of the same process, running on the same CPU core
 - iii) from one process to the same process, running on another CPU core
 - iv) from one thread to another thread of the same process, running on another CPU core
 - e Describe how you would modify the OS scheduler to improve performance, given your answers under part (d) above.

The five parts carry, respectively, 20%, 20%, 30%, 20%, and 10% of the marks.

2a Describe the steps taken by an OS kernel when executing a fork system call.

b The following C program uses the fork system call:

```
1  #include <unistd.h>
2  #include <stdio.h>

4  int main() {
5      int pid = fork();

7      if (pid != 0) {
8          printf("A");
9          pid = fork();
10     } else {
11         printf("B");
12         pid = fork();
13     }

15     if (pid != 0) {
16         printf("C");
17     } else
18         printf("D");

20     return 0;
21 }
```

- i) Draw a diagram that shows the process hierarchy corresponding to the above program when it is executed. For each process in the process hierarchy, list the lines that the process executes.
 - ii) Write down **four** possible different strings that the above program may print when executed repeatedly.
 - iii) Can the program ever output the string BCCADD? Justify your answer.
 - iv) State one line of code that can be added to the above program to ensure that B is always printed before A. Briefly explain your solution. (You can ignore error checking.)
- c You notice that a database application that heavily uses the fork system call exhibits poor performance. Give a possible cause of this performance problem and suggest an extension to the fork system call to avoid this problem.

The three parts carry, respectively, 15%, 70%, and 15% of the marks.